you can work with in your application

If your query is designed to return data, EF Core takes care of converting the data from the database into your domain objects. If you are familiar with software design patterns, you may well recognize this as an implementation of the Repository pattern (http://mng.bz/QnYv).

The SQL generated by EF Core depends on the provider you use. Each database system has its own provider, so in theory, the generated SQL should be optimized for the specific database. EF Core hides all of this away from your application code, so your LINQ query will work without amendment if you need to change the provider at some point. While migrations from one actual database system to another are rare in the real world, this capability is more useful if you want to replace your physical database with an in-memory database for testing purposes.

The approach you take when working with EF Core is known as *code first* (as opposed to *database first*), meaning you concentrate your efforts on developing the application model and allow EF Core to use the model as the basis for maintaining the database schema, using a concept known as *migrations*. EF Core can also create the database itself if it doesn't exist. EF Core relies on several conventions to map objects and their properties to database tables and columns and create relationships. You will explore the most important conventions as you refine your domain model to work with EF Core. In addition to the conventions that "just work," EF Core provides extensive configuration options that enable you to control how your model maps to tables and columns in the database.

8.1.3 Managing relationships